CS3014 - Estructuras de Datos Avanzadas

Jerarquía de Memoria IV
Semana 11

Víctor Racsó Galván Oyola

vgalvan@utec.edu.pe

20 de octubre de 2026

Objetivos de la sesión



Objetivos de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Abrir la caja negra del cache-oblivious B-Tree: el problema de Ordered File Maintenance

Objetivos de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Abrir la caja negra del cache-oblivious B-Tree: el problema de Ordered File Maintenance
- Construir el Packed-Memory Array, con umbrales de densidad que dependen de la profundidad

Objetivos de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Abrir la caja negra del cache-oblivious B-Tree: el problema de Ordered File Maintenance
- Construir el Packed-Memory Array, con umbrales de densidad que dependen de la profundidad
- Demostrar que cada actualización mueve $O(\lg^2 N)$ celdas consecutivas, amortizado

Objetivos de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Abrir la caja negra del cache-oblivious B-Tree: el problema de Ordered File Maintenance
- Construir el Packed-Memory Array, con umbrales de densidad que dependen de la profundidad
- Demostrar que cada actualización mueve $O(\lg^2 N)$ celdas consecutivas, amortizado
- Relacionarlo con List Labeling y resolver Order Maintenance en $O(1)$ con indirección

Motivación

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

La deuda del cache-oblivious B-Tree

Toda la versión dinámica se apoyó en una promesa: se puede mantener un arreglo ordenado, con huecos, reacomodando solo $O(\lg^2 N)$ celdas **consecutivas** por inserción.

Motivación

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

La deuda del cache-oblivious B-Tree

Toda la versión dinámica se apoyó en una promesa: se puede mantener un arreglo ordenado, con huecos, reacomodando solo $O(\lg^2 N)$ celdas **consecutivas** por inserción.

¿Por qué es difícil?

Un arreglo ordenado es la estructura con mejor localidad que existe, y la peor para insertar: un elemento nuevo en medio empuja a todos los que vienen después.

Motivación

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

La deuda del cache-oblivious B-Tree

Toda la versión dinámica se apoyó en una promesa: se puede mantener un arreglo ordenado, con huecos, reacomodando solo $O(\lg^2 N)$ celdas **consecutivas** por inserción.

¿Por qué es difícil?

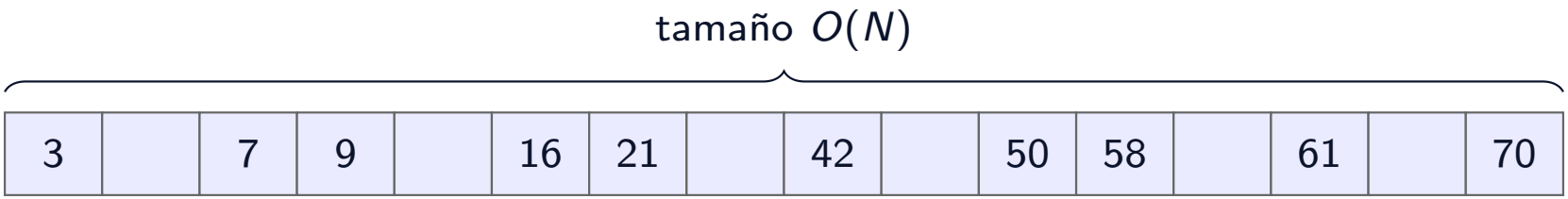
Un arreglo ordenado es la estructura con mejor localidad que existe, y la peor para insertar: un elemento nuevo en medio empuja a todos los que vienen después.

Hoy

Una estructura que conserva el orden físico y, aun así, inserta barato. Y dos problemas hermanos que se resuelven con las mismas ideas.

El problema

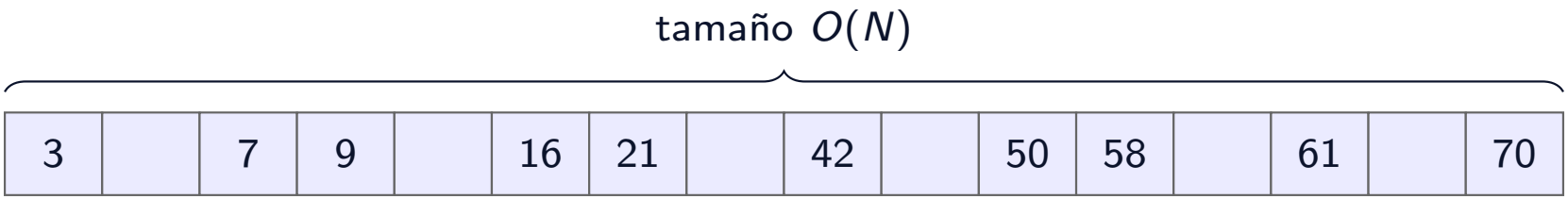
Ordered File Maintenance



Mantener N elementos en un arreglo, en orden, con

- tamaño total $O(N)$ y huecos de $O(1)$ celdas entre elementos consecutivos;

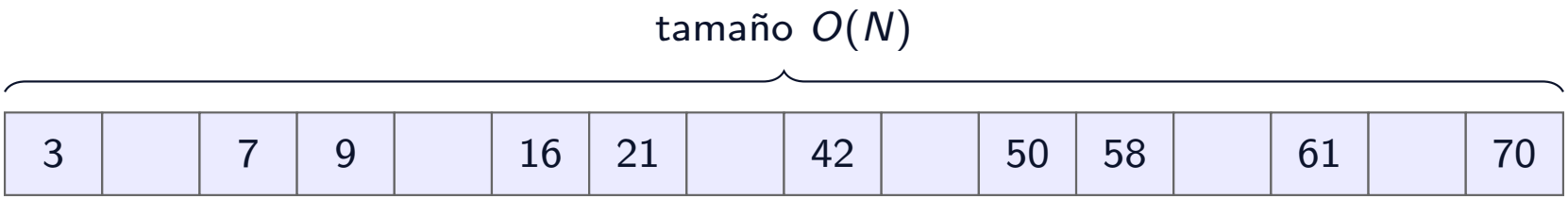
Ordered File Maintenance



Mantener N elementos en un arreglo, en orden, con

- tamaño total $O(N)$ y huecos de $O(1)$ celdas entre elementos consecutivos;
- INSERT: agregar un elemento entre dos elementos consecutivos dados;
- DELETE: quitar un elemento.

Ordered File Maintenance



Mantener N elementos en un arreglo, en orden, con

- tamaño total $O(N)$ y huecos de $O(1)$ celdas entre elementos consecutivos;
- INSERT: agregar un elemento entre dos elementos consecutivos dados;
- DELETE: quitar un elemento.

Los huecos pequeños garantizan que recorrer k elementos consecutivos siga costando $O(1 + k/B)$ transferencias. El orden en memoria es el orden de los datos.

Dos intentos que no funcionan

arreglo compacto

3	7	9	16	21	42	50	58								

un hueco por medio

3		7		9		16		21		42		50		58	

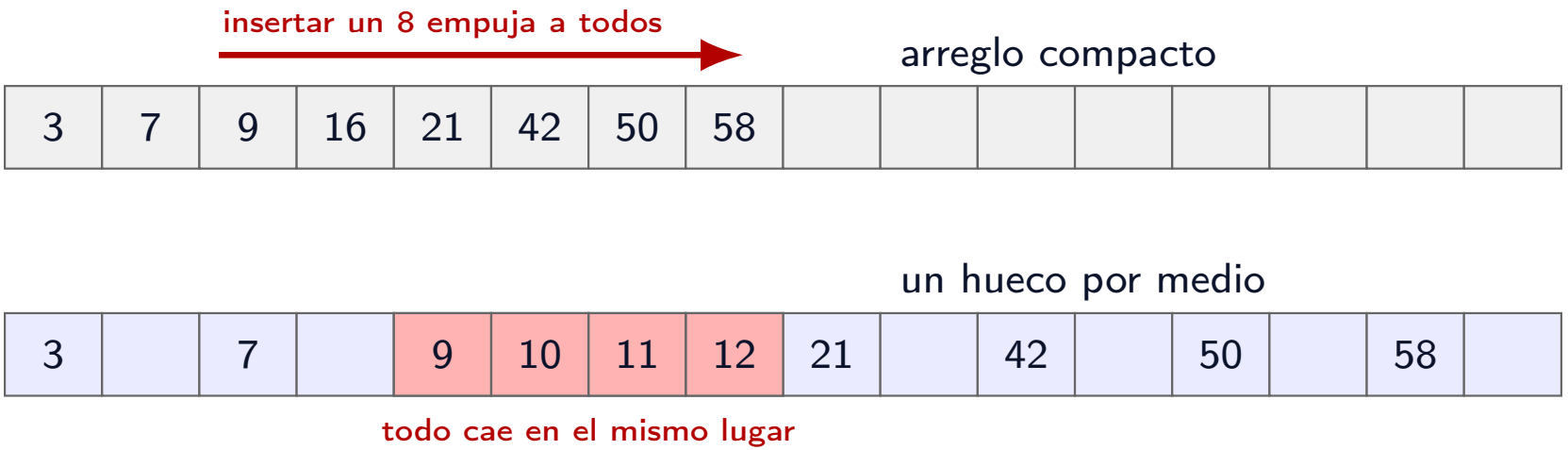
Dos intentos que no funcionan



Sin huecos

Insertar al inicio desplaza $\Theta(N)$ elementos.

Dos intentos que no funcionan



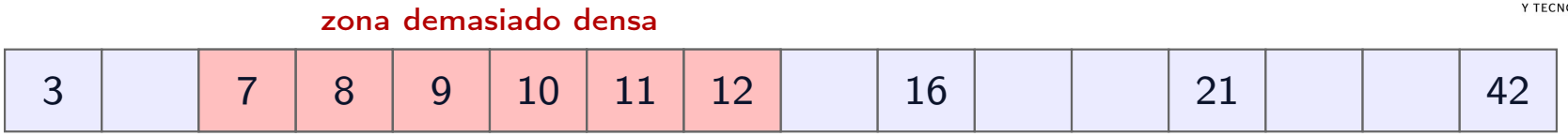
Sin huecos

Insertar al inicio desplaza $\Theta(N)$ elementos.

Con huecos repartidos de forma uniforme

Ayuda una vez. Si las inserciones insisten en la misma zona, los huecos de ahí se acaban y volvemos al caso anterior.

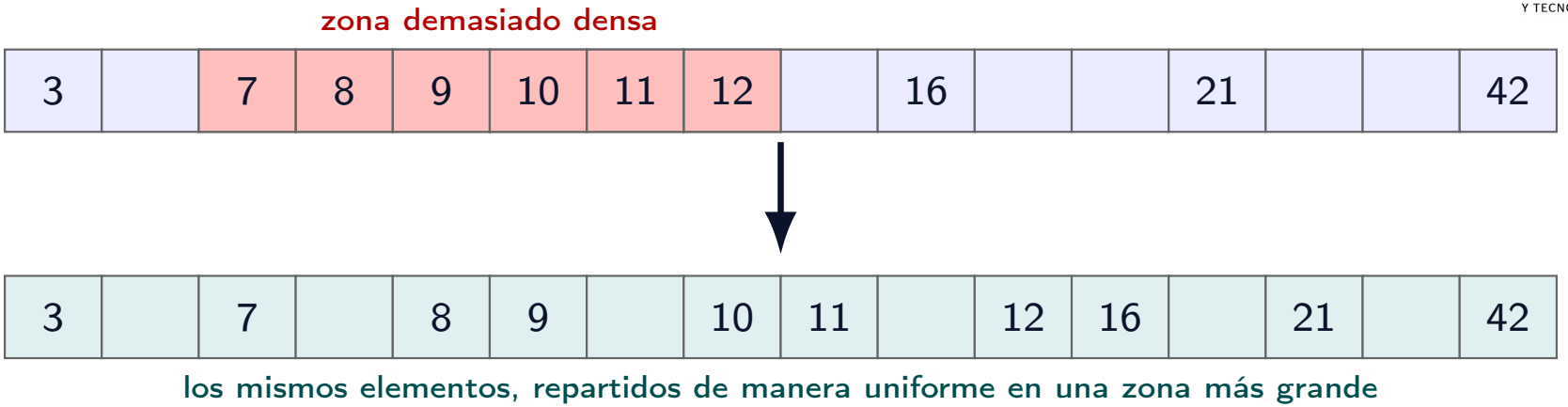
La idea: controlar la densidad



Densidad de un intervalo

$$\text{densidad} = \frac{\text{elementos en el intervalo}}{\text{celdas del intervalo}}$$

La idea: controlar la densidad



Densidad de un intervalo

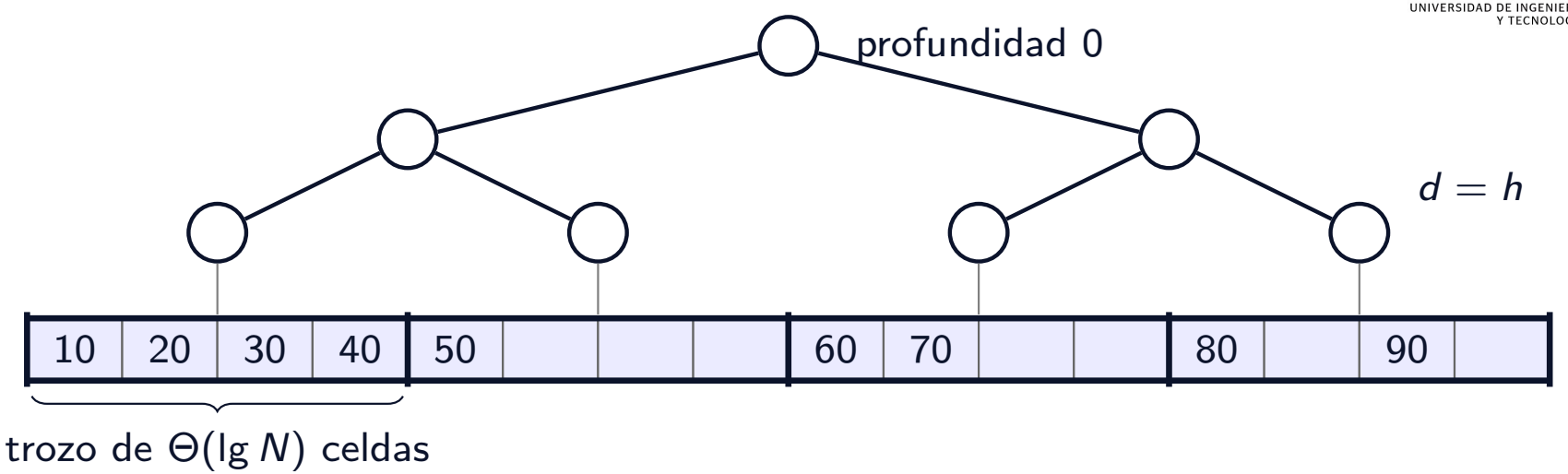
$$\text{densidad} = \frac{\text{elementos en el intervalo}}{\text{celdas del intervalo}}$$

La regla

Ningún intervalo debe quedar demasiado lleno ni demasiado vacío. Cuando uno se pasa, se reparten sus elementos de forma pareja en un intervalo **más grande** que lo contenga.

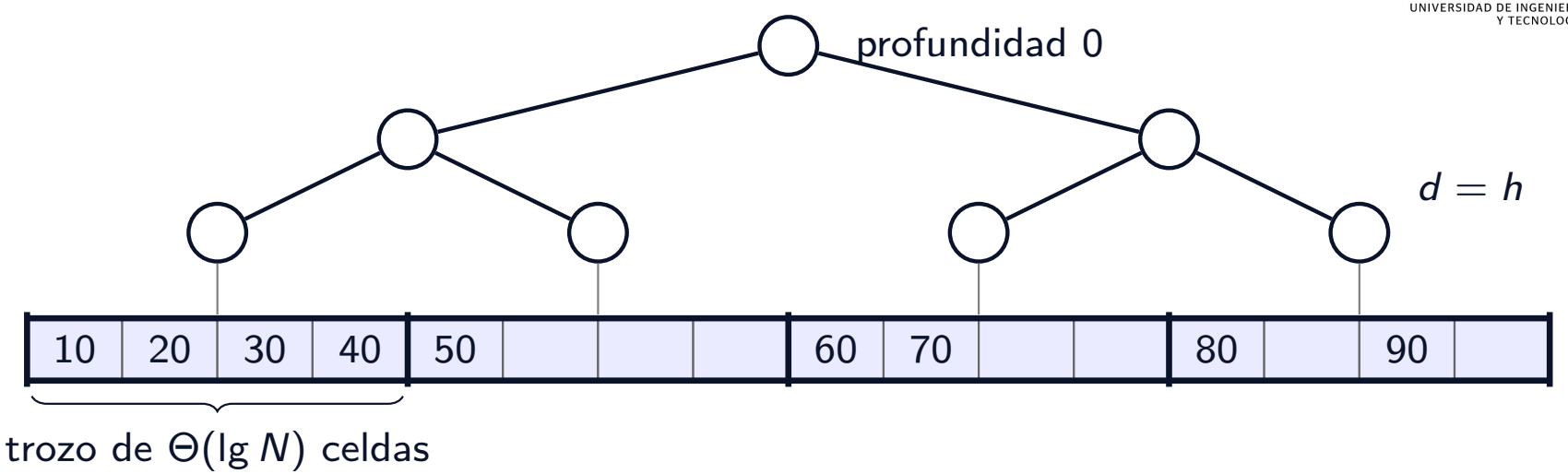
Packed-Memory Array

La estructura



Un arreglo y un árbol imaginario

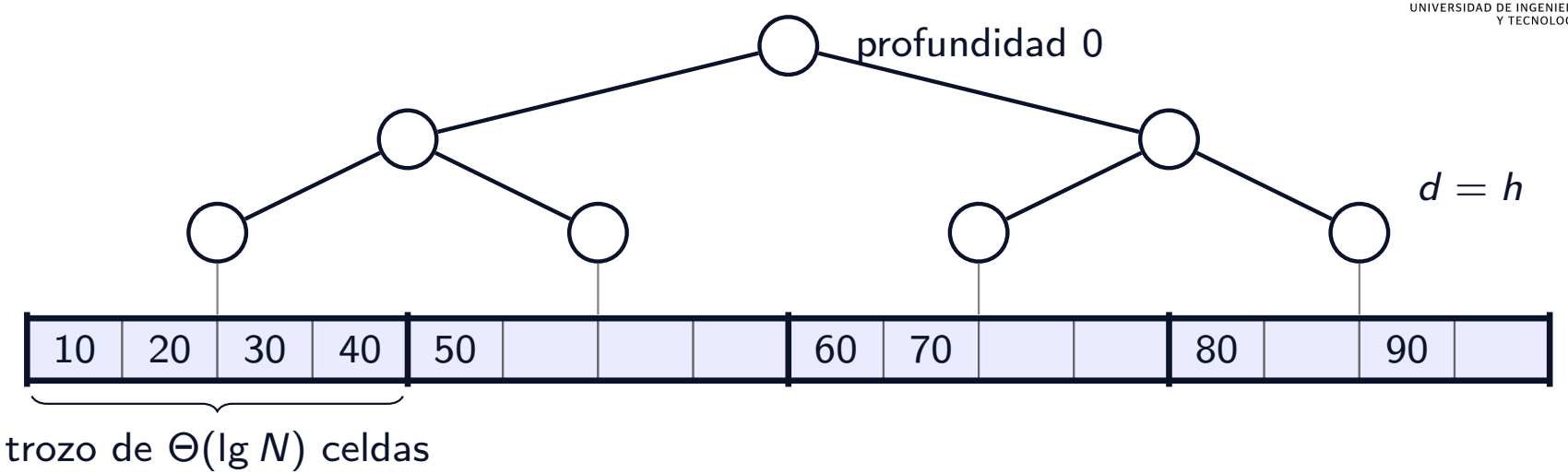
La estructura



Un arreglo y un árbol imaginario

El arreglo se divide en trozos de $\Theta(\lg N)$ celdas. Sobre los trozos imaginamos un árbol binario completo: cada nodo representa un intervalo del arreglo, y la raíz, el arreglo entero.

La estructura

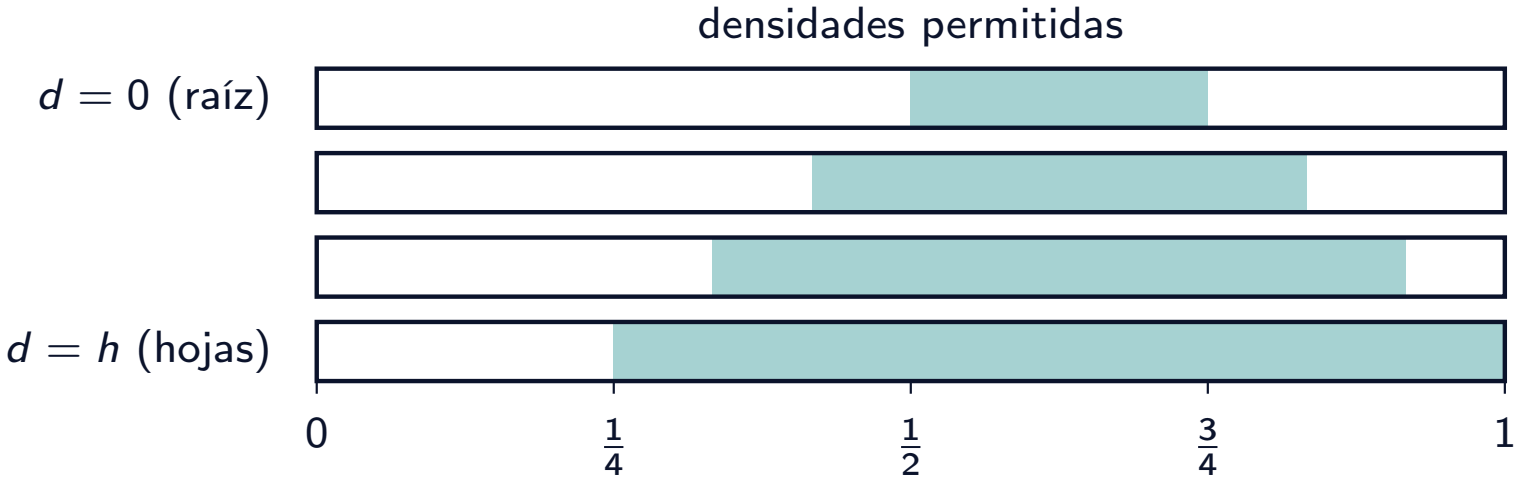


Un arreglo y un árbol imaginario

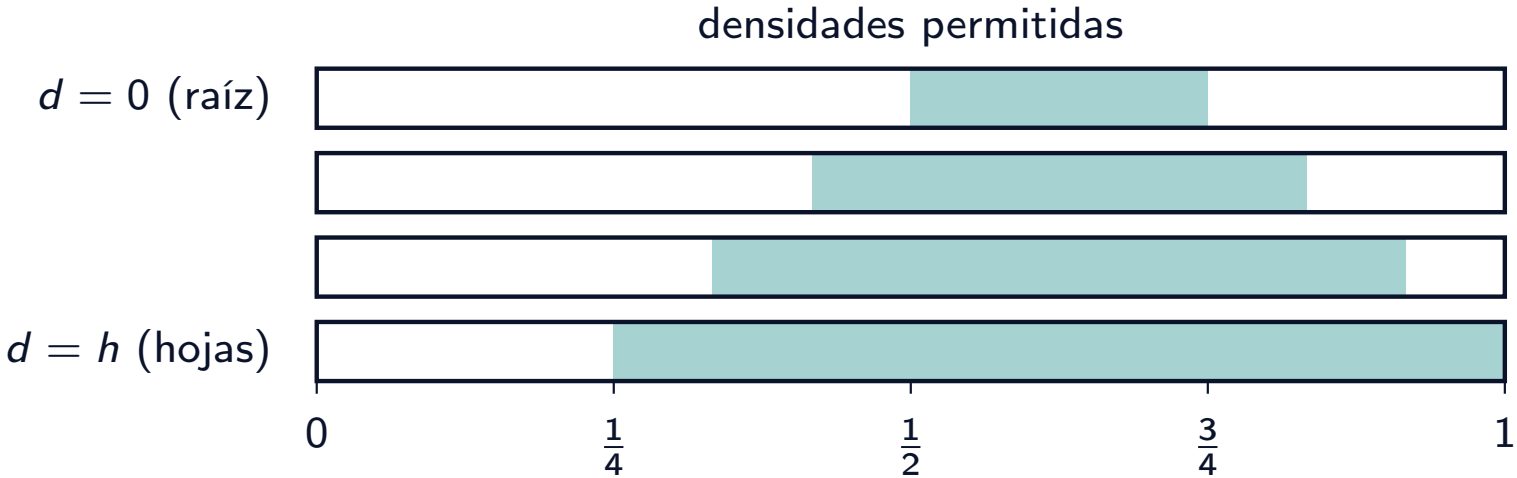
El arreglo se divide en trozos de $\Theta(\lg N)$ celdas. Sobre los trozos imaginamos un árbol binario completo: cada nodo representa un intervalo del arreglo, y la raíz, el arreglo entero.

La altura del árbol es $h = \Theta(\lg N)$. Cada nodo conoce cuántos elementos y cuántas celdas tiene su intervalo.

Umbrales que dependen de la profundidad



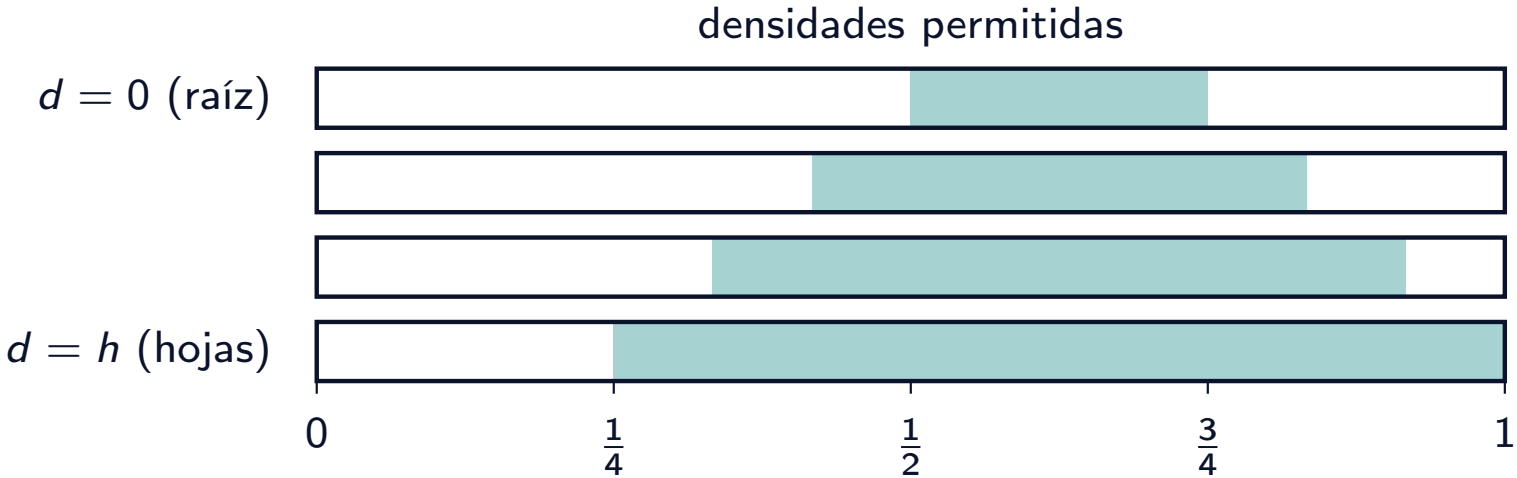
Umbrales que dependen de la profundidad



Para un nodo a profundidad d

$$\frac{1}{2} - \frac{1}{4} \cdot \frac{d}{h} \leq \text{densidad} \leq \frac{3}{4} + \frac{1}{4} \cdot \frac{d}{h}$$

Umbrales que dependen de la profundidad



Para un nodo a profundidad d

$$\frac{1}{2} - \frac{1}{4} \cdot \frac{d}{h} \leq \text{densidad} \leq \frac{3}{4} + \frac{1}{4} \cdot \frac{d}{h}$$

La raíz debe estar entre $\frac{1}{2}$ y $\frac{3}{4}$ de llena. Una hoja puede estar entre $\frac{1}{4}$ y completamente llena. Entre un nivel y el siguiente, cada umbral se mueve $\frac{1}{4h}$.

¿Por qué más estrictos arriba?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Después de repartir un intervalo de forma pareja

Todos sus descendientes quedan con la **misma** densidad que él, que cumple el umbral estricto del ancestro.

¿Por qué más estrictos arriba?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Después de repartir un intervalo de forma pareja

Todos sus descendientes quedan con la **misma** densidad que él, que cumple el umbral estricto del ancestro.

Entonces los descendientes quedan con holgura

Un hijo tiene un umbral $\frac{1}{4h}$ más permisivo que su padre. Recién redistribuido, está al menos a esa distancia de violar el suyo: tienen que llegar muchas inserciones antes de que vuelva a dar problemas.

Esa holgura es la que va a pagar el costo de las redistribuciones.

Insertar

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

El algoritmo

1. Insertar el elemento en su trozo, reescribiéndolo ($\Theta(\lg N)$ celdas).

Insertar

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

El algoritmo

1. Insertar el elemento en su trozo, reescribiéndolo ($\Theta(\lg N)$ celdas).
2. Si el trozo quedó dentro de su umbral, terminar.
3. Si no, subir por el árbol hasta el primer ancestro cuyo intervalo **sí** esté dentro de su umbral.

Insertar

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

El algoritmo

1. Insertar el elemento en su trozo, reescribiéndolo ($\Theta(\lg N)$ celdas).
2. Si el trozo quedó dentro de su umbral, terminar.
3. Si no, subir por el árbol hasta el primer ancestro cuyo intervalo **sí** esté dentro de su umbral.
4. Repartir de forma pareja todos los elementos del intervalo de ese ancestro.

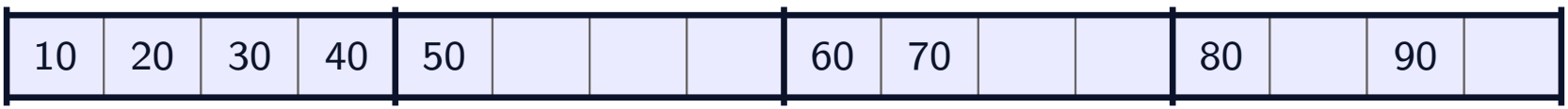
Insertar

El algoritmo

1. Insertar el elemento en su trozo, reescribiéndolo ($\Theta(\lg N)$ celdas).
2. Si el trozo quedó dentro de su umbral, terminar.
3. Si no, subir por el árbol hasta el primer ancestro cuyo intervalo **sí** esté dentro de su umbral.
4. Repartir de forma pareja todos los elementos del intervalo de ese ancestro.

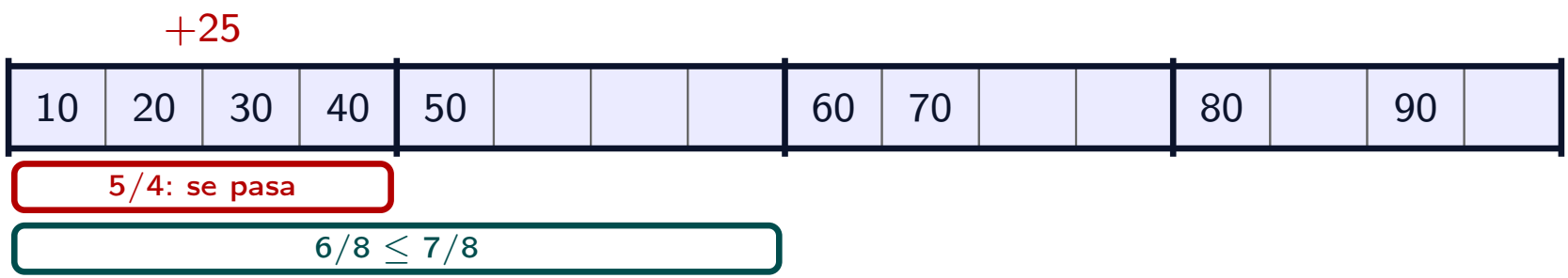
Cuanto más arriba haya que subir, más grande es el intervalo que se reescribe.
Eliminar es igual, con los umbrales inferiores.

Insertar, en un ejemplo



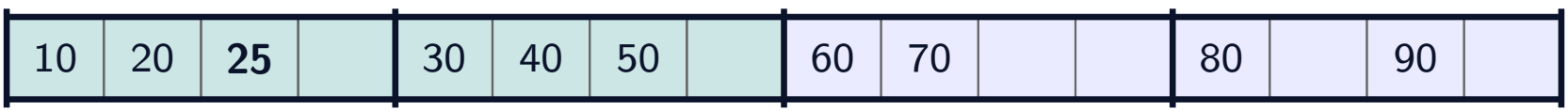
16 celdas en trozos de 4; $h = 2$. Umbrales superiores: 1 en las hojas, $\frac{7}{8}$ en el nivel medio, $\frac{3}{4}$ en la raíz.

Insertar, en un ejemplo



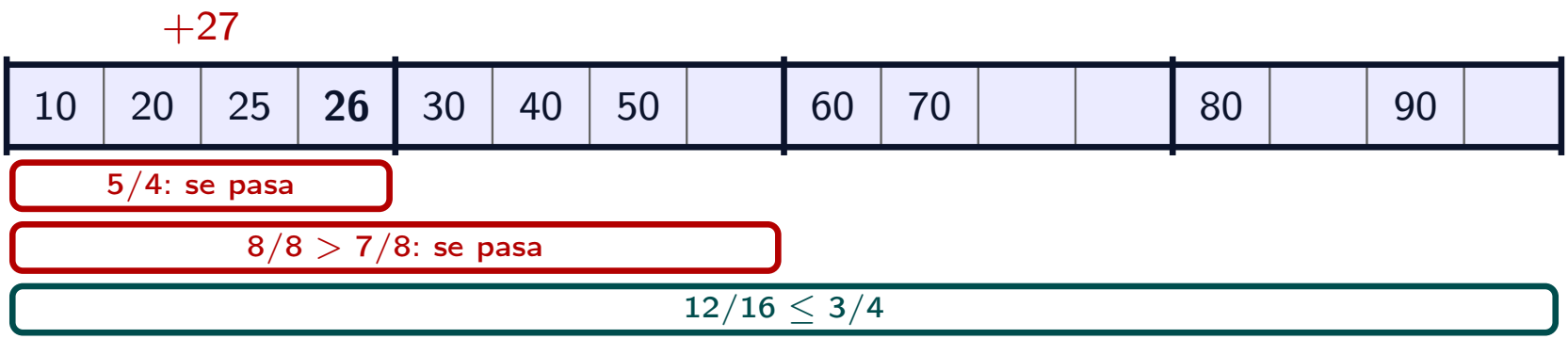
Insertar 25: su trozo quedaría con 5 elementos. Su padre, con 6 de 8, está dentro del umbral.

Insertar, en un ejemplo



Se reparten los 6 elementos en las 8 celdas del padre.

Insertar, en un ejemplo



El 26 entra sin problemas. Con el 27 se pasan el trozo y también su padre; la raíz, con 12 de 16, aguanta.

Insertar, en un ejemplo



Se reparte todo el arreglo: tres elementos por trozo.

El árbol no existe

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Es una herramienta para pensar

No se guarda ningún nodo. Un "nodo" es un intervalo del arreglo cuyos extremos se calculan con aritmética.

El árbol no existe

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Es una herramienta para pensar

No se guarda ningún nodo. Un "nodo" es un intervalo del arreglo cuyos extremos se calculan con aritmética.

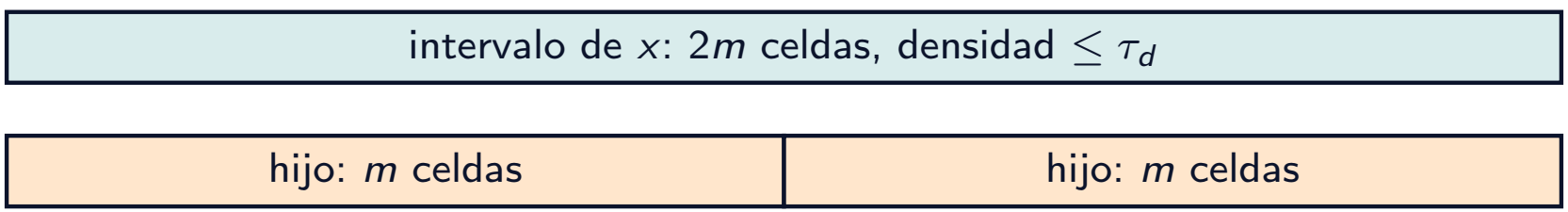
Subir es agrandar el intervalo

Se parte del trozo y se va duplicando el intervalo hacia el lado que corresponda, contando elementos con un escaneo. Al encontrar el intervalo que cumple, se reparten sus elementos con otro par de escaneos. Todo son recorridos de celdas consecutivas: si el intervalo tiene K celdas, cuesta $O(1 + K/B)$ transferencias.

¿Cuánto cuesta?

En el peor caso, una inserción reescribe todo el arreglo

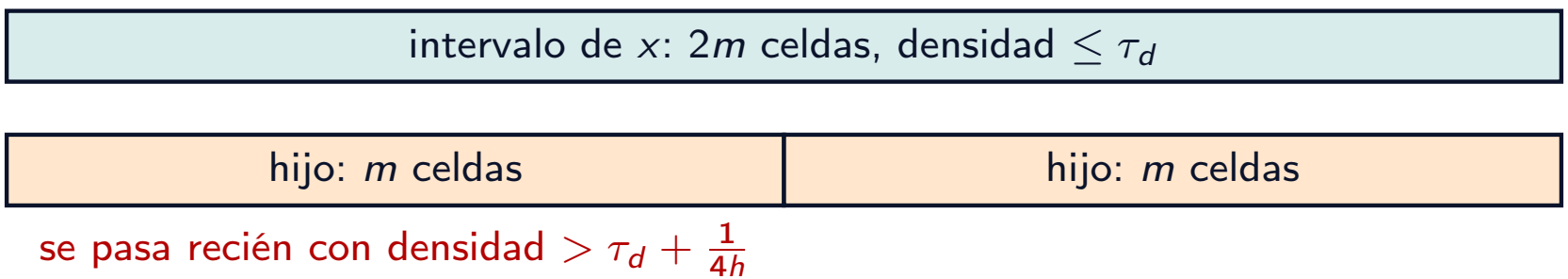
Pero no puede pasar seguido. Miremos un nodo x a profundidad d, con un intervalo de $2m$ celdas, justo después de redistribuirlo.



¿Cuánto cuesta?

En el peor caso, una inserción reescribe todo el arreglo

Pero no puede pasar seguido. Miremos un nodo x a profundidad d, con un intervalo de $2m$ celdas, justo después de redistribuirlo.



Los hijos tardan en volver a pasarse

Cada hijo quedó con densidad $\leq \tau_d$, y su umbral es $\tau_{d+1} = \tau_d + \frac{1}{4h}$. Para superarlo hacen falta más de $\frac{m}{4h}$ inserciones dentro de **ese** hijo.

El argumento de cobro

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

¿Quién paga la redistribución de x?

Se redistribuye x porque uno de sus hijos se pasó de su umbral. Eso costó $2m$ movimientos, y antes hubo al menos $\frac{m}{4h}$ actualizaciones en ese hijo desde la última vez.

El argumento de cobro

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

¿Quién paga la redistribución de x?

Se redistribuye x porque uno de sus hijos se pasó de su umbral. Eso costó $2m$ movimientos, y antes hubo al menos $\frac{m}{4h}$ actualizaciones en ese hijo desde la última vez.

Se lo cobramos a esas actualizaciones

$$\frac{2m}{m/(4h)} = 8h = O(\lg N) \quad \text{por actualización.}$$

El argumento de cobro

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

¿Quién paga la redistribución de x?

Se redistribuye x porque uno de sus hijos se pasó de su umbral. Eso costó $2m$ movimientos, y antes hubo al menos $\frac{m}{4h}$ actualizaciones en ese hijo desde la última vez.

Se lo cobramos a esas actualizaciones

$$\frac{2m}{m/(4h)} = 8h = O(\lg N) \quad \text{por actualización.}$$

¿Y cuántas veces se le cobra a una misma actualización?

Una actualización cae dentro de un intervalo por cada nivel del árbol: a lo más $h = O(\lg N)$ cobros de $O(\lg N)$ cada uno.

El resultado

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Teorema (Packed-Memory Array)

INSERT y DELETE mueven $O(\lg^2 N)$ celdas consecutivas, amortizado, mediante un número constante de escaneos: $O\left(1 + \frac{\lg^2 N}{B}\right)$ transferencias.

El resultado

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Teorema (Packed-Memory Array)

INSERT y DELETE mueven $O(\lg^2 N)$ celdas consecutivas, amortizado, mediante un número constante de escaneos: $O\left(1 + \frac{\lg^2 N}{B}\right)$ transferencias.

Lo que faltaba

Si la raíz misma se sale de $[\frac{1}{2}, \frac{3}{4}]$, el arreglo completo está muy lleno o muy vacío: se reconstruye con el doble o la mitad de tamaño, como una tabla dinámica.

El resultado

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Teorema (Packed-Memory Array)

INSERT y DELETE mueven $O(\lg^2 N)$ celdas consecutivas, amortizado, mediante un número constante de escaneos: $O\left(1 + \frac{\lg^2 N}{B}\right)$ transferencias.

Lo que faltaba

Si la raíz misma se sale de $[\frac{1}{2}, \frac{3}{4}]$, el arreglo completo está muy lleno o muy vacío: se reconstruye con el doble o la mitad de tamaño, como una tabla dinámica.

La cota se puede lograr en el peor caso, no solo amortizada. Y se conjetura que $\lg^2 N$ es lo mejor posible para este problema.

List Labeling

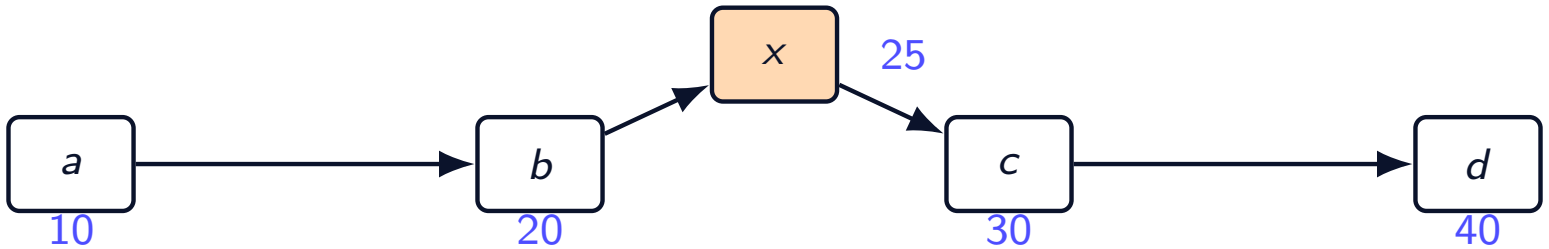
Un problema hermano: List Labeling



El problema

Mantener una lista enlazada en la que cada nodo lleva una **etiqueta** entera, creciente a lo largo de la lista.

Un problema hermano: List Labeling



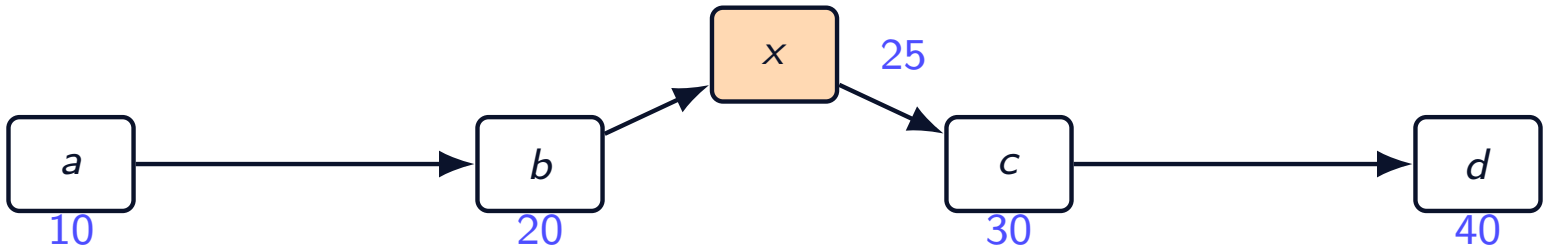
El problema

Mantener una lista enlazada en la que cada nodo lleva una **etiqueta** entera, creciente a lo largo de la lista.

Insertar entre dos nodos

El nuevo recibe una etiqueta intermedia. Si no queda ninguna, hay que **reetiquetar** vecinos. El costo es cuántas etiquetas cambian.

Un problema hermano: List Labeling



El problema

Mantener una lista enlazada en la que cada nodo lleva una **etiqueta** entera, creciente a lo largo de la lista.

Insertar entre dos nodos

El nuevo recibe una etiqueta intermedia. Si no queda ninguna, hay que **reetiquetar** vecinos. El costo es cuántas etiquetas cambian.

Ordered File Maintenance es List Labeling: la etiqueta de un elemento es su **posición** en el arreglo.

El espacio de etiquetas lo cambia todo

Etiquetas	Costo	Cómo
Lineal, hasta $N \lg N$	$O(\lg^2 N)$	Packed-Memory Array

El espacio de etiquetas lo cambia todo

Etiquetas	Costo	Cómo
Lineal, hasta $N \lg N$	$O(\lg^2 N)$	Packed-Memory Array
Polinomial, $N^{O(1)}$	$\Theta(\lg N)$	La misma idea, más holgura

Con más etiquetas, los umbrales se relajan

Si el "arreglo" no tiene que existir físicamente, puede ser mucho más grande que N, y hay que redistribuir con menos frecuencia.

El espacio de etiquetas lo cambia todo

Etiquetas	Costo	Cómo
Lineal, hasta $N \lg N$	$O(\lg^2 N)$	Packed-Memory Array
Polinomial, $N^{O(1)}$	$\Theta(\lg N)$	La misma idea, más holgura
Exponencial, 2^N	$\Theta(1)$	Punto medio

Con más etiquetas, los umbrales se relajan

Si el "arreglo" no tiene que existir físicamente, puede ser mucho más grande que N, y hay que redistribuir con menos frecuencia.

El caso exponencial

Con N bits por etiqueta, el nuevo toma el punto medio entre sus vecinos. Siempre hay lugar durante N inserciones; luego se reetiqueta todo.

Order Maintenance

Order Maintenance

UTEC
UNIVERSIDAD DE INGENIERÍA

El problema

Mantener una lista bajo inserciones y eliminaciones, y responder consultas $\textsc{Order}(x, y)$: ¿está x antes que y en la lista?

Order Maintenance

UTEC
UNIVERSIDAD DE INGENIERÍA

El problema

Mantener una lista bajo inserciones y eliminaciones, y responder consultas $\textsc{Order}(x, y)$: ¿está x antes que y en la lista?

Con etiquetas, la consulta es una comparación

Si las etiquetas crecen a lo largo de la lista, $\textsc{Order}(x, y)$ es comparar dos enteros: $O(1)$. Con etiquetas polinomiales (que caben en una palabra), las actualizaciones cuestan $O(\lg N)$.

Order Maintenance

UTEC
UNIVERSIDAD DE INGENIERÍA

El problema

Mantener una lista bajo inserciones y eliminaciones, y responder consultas $\textsc{Order}(x, y)$: ¿está x antes que y en la lista?

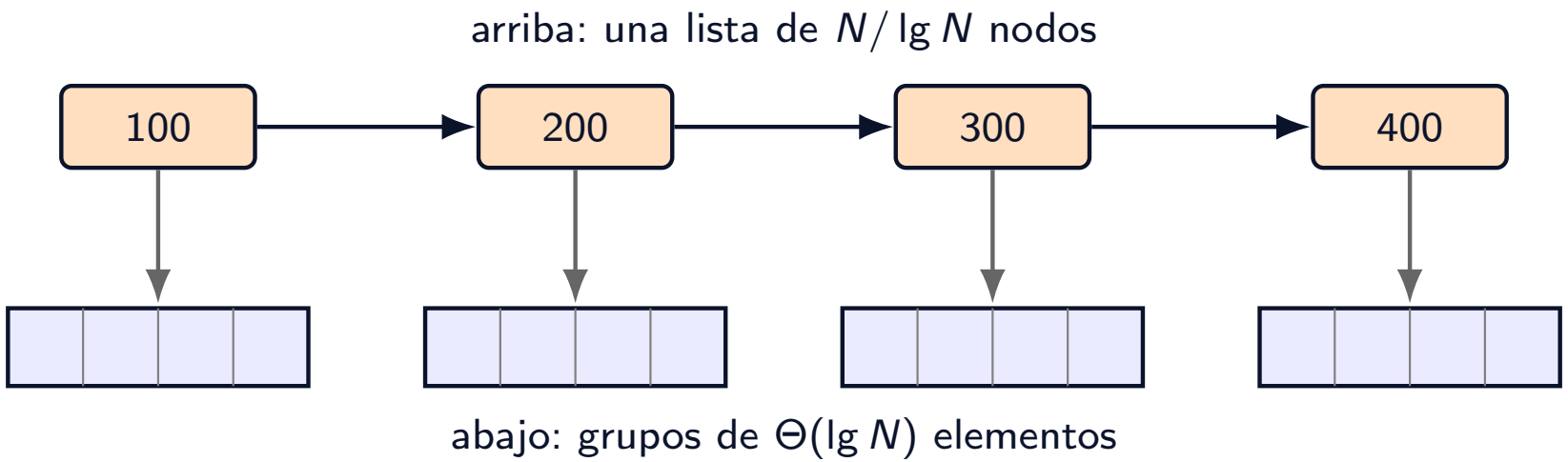
Con etiquetas, la consulta es una comparación

Si las etiquetas crecen a lo largo de la lista, $\textsc{Order}(x, y)$ es comparar dos enteros: $O(1)$. Con etiquetas polinomiales (que caben en una palabra), las actualizaciones cuestan $O(\lg N)$.

¿Dónde apareció esto antes?

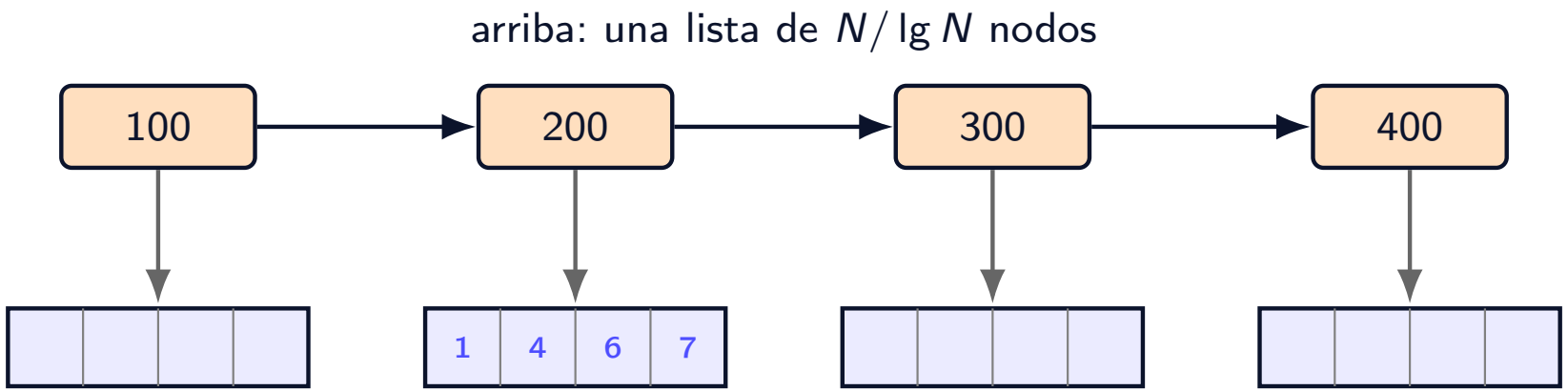
En persistencia total: para decidir si una versión es ancestro de otra en el árbol de versiones, lo linealizamos en una lista y necesitábamos comparar posiciones. Ahí lo usamos como caja negra. ¿Se puede en $O(1)$ también al actualizar?

Indirección, otra vez



Dos niveles de etiquetas

Indirección, otra vez



Dos niveles de etiquetas

Arriba, una lista con un nodo por grupo, etiquetada con el método polinomial.

Abajo, cada grupo de $\Theta(\lg N)$ elementos usa el método exponencial: necesita $2^{\Theta(\lg N)} = N^{O(1)}$ etiquetas, que caben en una palabra.

¿Por qué todo cuesta $O(1)$?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Consultar

$\textsc{Order}(x, y)$ compara los pares (arriba, abajo) en orden lexicográfico: dos comparaciones.

¿Por qué todo cuesta $O(1)$?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Consultar

$\textsc{Order}(x, y)$ compara los pares (arriba, abajo) en orden lexicográfico: dos comparaciones.

Actualizar

Insertar dentro de un grupo cuesta $O(1)$ amortizado. Solo cuando un grupo se llena o se vacía se parte o se fusiona, y recién entonces se toca la lista de arriba, a costo $O(\lg N)$. Eso pasa una vez cada $\Omega(\lg N)$ actualizaciones del grupo: $O(1)$ amortizado.

¿Por qué todo cuesta $O(1)$?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Consultar

$\textsc{Order}(x, y)$ compara los pares (arriba, abajo) en orden lexicográfico: dos comparaciones.

Actualizar

Insertar dentro de un grupo cuesta $O(1)$ amortizado. Solo cuando un grupo se llena o se vacía se parte o se fusiona, y recién entonces se toca la lista de arriba, a costo $O(\lg N)$. Eso pasa una vez cada $\Omega(\lg N)$ actualizaciones del grupo: $O(1)$ amortizado.

La clave es que cambiar **una** etiqueta de arriba cambia, de golpe y sin tocarlos, el orden relativo de los $\Theta(\lg N)$ elementos de su grupo. Es la misma indirección que usamos en el cache-oblivious B-Tree.

El mapa de la unidad

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Tema	Resultado
Modelo de memoria externa	Escanear N/B; ordenar $\frac{N}{B} \log_{M/B} \frac{N}{B}$
B-Trees	Buscar y actualizar en $\log_{B+1} N$, conociendo B

El mapa de la unidad

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Tema	Resultado
Modelo de memoria externa	Escanear N/B; ordenar $\frac{N}{B} \log_{M/B} \frac{N}{B}$
B-Trees	Buscar y actualizar en $\log_{B+1} N$, conociendo B
Modelo cache-oblivious	Las mismas cotas sin conocer B ni M
Layout de van Emde Boas	Buscar en $\log_{B+1} N$ para todo B

El mapa de la unidad

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Tema	Resultado
Modelo de memoria externa	Escanear N/B; ordenar $\frac{N}{B} \log_{M/B} \frac{N}{B}$
B-Trees	Buscar y actualizar en $\log_{B+1} N$, conociendo B
Modelo cache-oblivious	Las mismas cotas sin conocer B ni M
Layout de van Emde Boas	Buscar en $\log_{B+1} N$ para todo B
Packed-Memory Array	Insertar en un arreglo ordenado moviendo $\lg^2 N$ celdas
Indirección	Quitar factores $\lg N$: B-Tree dinámico, Order Maintenance en $O(1)$

Cierre de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Ordered File Maintenance: un arreglo ordenado con huecos de $O(1)$, que admite inserciones y eliminaciones.
- Packed-Memory Array: trozos de $\Theta(\lg N)$, un árbol imaginario de intervalos y umbrales de densidad más estrictos cerca de la raíz.

Cierre de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Ordered File Maintenance: un arreglo ordenado con huecos de $O(1)$, que admite inserciones y eliminaciones.
- Packed-Memory Array: trozos de $\Theta(\lg N)$, un árbol imaginario de intervalos y umbrales de densidad más estrictos cerca de la raíz.
- Cada redistribución se cobra a las actualizaciones que agotaron la holgura del hijo: $O(\lg N)$ por nivel y $O(\lg^2 N)$ en total, amortizado.
- List Labeling generaliza el problema; con más etiquetas, menos costo. Order Maintenance se resuelve en $O(1)$ con dos niveles de etiquetas.

Logros de la sesión



Logros de la sesión

- Sabemos ejecutar una inserción en un Packed-Memory Array, incluyendo la redistribución

Logros de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Sabemos ejecutar una inserción en un Packed-Memory Array, incluyendo la redistribución
- Sabemos justificar por qué los umbrales dependen de la profundidad y demostrar la cota $O(\lg^2 N)$

Logros de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Sabemos ejecutar una inserción en un Packed-Memory Array, incluyendo la redistribución
- Sabemos justificar por qué los umbrales dependen de la profundidad y demostrar la cota $O(\lg^2 N)$
- Sabemos relacionar Ordered File Maintenance con List Labeling y sus distintas cotas

Logros de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Sabemos ejecutar una inserción en un Packed-Memory Array, incluyendo la redistribución
- Sabemos justificar por qué los umbrales dependen de la profundidad y demostrar la cota $O(\lg^2 N)$
- Sabemos relacionar Ordered File Maintenance con List Labeling y sus distintas cotas
- Sabemos resolver Order Maintenance en $O(1)$ con indirección

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Gracias

Referencias

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Bibliografía citada

- Itai, A., Konheim, A. G., & Rodeh, M. (1981). A sparse table implementation of priority queues. *Proceedings of ICALP'81*, 417–431.
- Bender, M. A., Cole, R., Demaine, E. D., Farach-Colton, M., & Zito, J. (2002). Two simplified algorithms for maintaining order in a list. *Proceedings of ESA'02*, 152–164.
- Dietz, P., & Sleator, D. (1987). Two algorithms for maintaining order in a list. *Proceedings of STOC'87*, 365–372.
- Demaine, Erik (2021). *6.851: Advanced Data Structures (Spring'21)*. `https://courses.csail.mit.edu/6.851/spring21/`.

Agradecimientos

Estas diapositivas están basadas en las clases de Luciano A. Romero Calla para este curso (a su vez basadas en las clases de Erik Demaine, MIT 6.851), así como en clases similares de otras universidades.
Lista de créditos: Luciano A. Romero Calla, Erik Demaine, Keith Schwarz.